Multi-Agent Collaboration Platform — 技术调研报告

> 调研日期：2026-04-13 目标：评估构建一个通用多 Agent 协作平台的技术路线 战略定位：通用平台，以游戏为初期切入点（赛道空白），架构需 future-proof 初期目标（难度递增）：圆桌辩论 → 狼人杀 → 剧本杀 → TRPG 远期场景：三省六部式 OPC、团队 brainstorming、教育模拟、客服演练、自定义场景

1. 产品愿景与战略

1.1 核心定位

这不是一个游戏平台，而是一个通用多 Agent 协作平台。 游戏（狼人杀、剧本杀、跑团）是初期切入点，因为： - 赛道极度空白（整个领域没有超过 ~100 stars 的项目） - 游戏天然需要多 Agent 最核心的能力：角色扮演、信息隔离、回合管理 - 游戏场景有天然的病毒传播属性（用户愿意分享、围观） - 游戏验证了平台核心后，可以自然扩展到严肃场景

1.2 用户体验愿景

用户可以： - **创建房间**，拉入多个 AI Agent（和可选的人类参与者） - **为每个 Agent 写人设**（简单几句话，Agent 自动丰富成完整角色） - **选择或自定义模式**（游戏 / 辩论 / 协作 / 自由聊天 / 自定义流程） - **观看 Agent 之间的互动**，群聊式可视化（气泡、@mentions、时间线） - **旁观者/上帝视角**模式 - 每个 Agent 可以选不同的底层模型（Claude、GPT、Gemini、Qwen 等）

1.3 架构约束（源自战略定位）

因为是通用平台而非纯游戏引擎，架构必须满足： 1. **模式可插拔** — "游戏规则引擎"是一个 Mode 插件，不是平台核心 2. **流程控制可扩展** — 自由聊天、回合制、状态机、层级委派都是流程的不同实现 3. **Agent 身份可持久化** — Agent 不只存在于一局游戏，而是一个可复用的"人格" 4. **通信模型足够通用** — 信息隔离不只为游戏服务，协作场景同样需要（分组讨论 → 汇总）

1.4 核心技术挑战

1. **通用通信层** — 频道/房间/可见性模型需要同时支撑游戏隔离和协作分组
2. **可插拔流程控制** — 从完全自由到严格状态机，需要统一的抽象
3. **角色人设 + 记忆** — 跨会话的持久人格，不只是局内记忆
4. **多 Agent 可视化 UI** — 这是用户体验的核心，也是 Accio Work 的杀手级优势

2. 调研范围

2.1 已评估的框架/平台

项目	类型	Stars	适合度	备注
AgentScope (agentscope-ai/agentscope)	Python multi-agent 框架	23.5k	⭐⭐⭐⭐	MsgHub 信息隔离、多模型支持、有狼人杀 sample
AgentScope Samples (agentscope-samples)	示例集	274	—	狼人杀（质量好）、evotraders（多 agent 可视化）、alias（全栈）
ClawTeam (HKUDS/ClawTeam)	代码协作编排框架	4.7k	❌	面向软件工程，无游戏概念
HiClaw (agentscope-ai/HiClaw)	Docker agent 运行时	4k	❌	容器编排层，不是游戏框架
ChatArena (Farama)	多 agent 语言游戏	1.5k	⭐⭐⭐⭐	架构设计最优雅，但已停更 (2025.8)
MetaGPT	多 agent 软件工程	67k	⭐⭐	有社区狼人杀 example，但核心面向 dev workflow
CAMEL	角色扮演 agent	16.7k	⭐⭐	研究导向，弱于实际游戏执行
Stanford Generative Agents	Agent 模拟	21k	⭐⭐⭐	记忆/反思/规划系统是金标准
Danghuangshang (当皇上)	三省六部 TS 实现	2.6k	⭐⭐⭐	TypeScript、有 Dashboard UI

2.2 已评估的垂直项目

项目	类型	Stars	亮点
ai-murder-mystery	AI 剧本杀	16	React + FastAPI，多 agent 人格+记忆，证据系统，多模型支持，有完整 Web UI
jubensha-ai	AI 剧本杀	89	最成熟的中文 AI 剧本杀项目，含 TTS + AI 生图
dnd-llm-game	AI 跑团	101	AI DM + AI party，Streamlit UI
Google werewolf_arena	狼人杀	46	Google 出品，小巧

2.3 闭源参考

产品	价值
Accio Work (阿里国际站)	UX 范式参考：群聊式多 Agent 交互、团队编排、多模型混用、可视化任务看板

3. AgentScope 深度评估

3.1 核心架构

```
AgentBase (async reply/observe)
  └─ ReActAgent (工具使用 + 结构化输出)
  └─ UserAgent (人类玩家)
  └─ A2AAgent (跨服务通信)

Msg (name, content, role, metadata, ContentBlock[])
  └─ TextBlock, ToolUseBlock, ImageBlock, ThinkingBlock...

MsgHub (信息隔离的核心)
  └─ 命名作用域，管理订阅者列表
  └─ auto_broadcast 开关：控制消息是否自动广播
  └─ 支持嵌套 hub (外层全体玩家 + 内层狼人专用)
  └─ 动态增删参与者

Pipeline
  └─ SequentialPipeline (回合制讨论)
  └─ FanoutPipeline (并行投票)

Memory
  └─ WorkingMemory (InMemory / Redis / SQLAlchemy)
  └─ LongTermMemory (mem0 / reme)
  └─ Session 持久化 (JSON / Redis / Tablestore)

LLM
  └─ OpenAI, Anthropic, Gemini, DashScope, Ollama, Trinity
  └─ 不同 Agent 可以用不同模型
```

3.2 狼人杀实现分析（agentscope-samples）

优点： - 全 async/await - Pydantic 结构化输出约束投票（`Literal[tuple(alive_players)]`，防幻觉） - 嵌套 MsgHub 实现信息隔离（狼人夜间讨论 vs 白天公屏） - `fanout_pipeline` 并行投票 + 事后广播 - JSONSession 支持跨局持久化

缺点： - 是脚本，不是框架 — 一个 ~400 行的 `game.py`，无状态机 - Moderator 是 EchoAgent（无 LLM），纯消息转发 - 没有人类玩家支持 - 投票无平票处理 - 不可扩展到其他游戏 — 需要重写所有游戏逻辑

3.3 UI 能力评估（来自 samples）

示例	多 Agent 可视化	通信方式	可复用性
evotraders	✅ 语音气泡 + Room View + Agent Feed	WebSocket	高 — AgentFeed, RoomView, ReadOnlyClient 可搬
alias	❌ 单 agent 对话	SSE	中 — Viewer 组件可搬
chatbot_fullstack	❌ 单 agent 对话	REST 轮询	低 — 单文件 demo

3.4 风险因素

1. **v2.0 正在重写** — core modules (model, agent, event, tools) 正在重构，API 即将 breaking change
2. **核心团队极小** — 2 人贡献 95% 代码，70% issues 无回复
3. **依赖较重** — 基础安装拉入 OpenAI + Anthropic + DashScope + OpenTelemetry + sounddevice
4. **Python ↔ JS 桥接** — 你的 UI 是 JS，AgentScope 是 Python，需要 WebSocket/SSE 桥
5. **游戏 example 极少** — 只有狼人杀，无剧本杀/跑团

4. 核心架构设计 — 平台优先，模式可插拔

4.1 关键洞察：好的多 Agent 协作 = 好的游戏体验

三省六部和狼人杀本质上用的是**同一套基础设施** — 多个 Agent 在受控的通信拓扑中按规则互动。区别只是"规则配置"不同：

平台核心能力	游戏场景	协作场景
信息隔离 (频道/可见性)	狼人身份、私密线索	分组讨论、保密审议
角色人设 (持久人格)	村民/狼人/侦探	财务官/战略官/法务
流程控制 (状态机)	夜晚→白天→投票	提案→审议→决策
旁观/回放	上帝视角	老板看团队讨论
结构化输出 (约束决策)	投票必须选真实玩家	审批必须选通过/驳回

把协作做好，游戏自然好。 平台核心不需要知道"狼人杀"是什么 — 它只需要知道如何管理 Agent、频道、消息可见性和流程控制。

4.2 三层架构

```
┌──────────────────────────────────────────────────────────────┐
│                    Layer 3: Mode 模式层                       │
│  ┌──────────┐ ┌──────────┐ ┌──────────┐ ┌──────────────┐    │
│  │ 圆桌辩论  │ │  狼人杀   │ │  剧本杀   │ │ TRPG / 自定义 │    │
│  │ FreeTalk │ │ Werewolf │ │ScriptKill│ │   TRPG / ...  │    │
│  └──────────┘ └──────────┘ └──────────┘ └──────────────┘    │
│  每个 Mode 提供：流程定义、角色模板、规则约束、UI 扩展点          │
├──────────────────────────────────────────────────────────────┤
│                  Layer 2: 平台核心 (mode-agnostic)            │
│                                                              │
│  ┌──────────────┐ ┌──────────────┐ ┌────────────────────────┐ │
│  │ Agent System │ │ Comm Layer   │ │  Flow Controller       │ │
│  │ - 人设定义    │ │ - Room       │ │  - FreeForm (自由聊天)  │ │
│  │ - 模型选择    │ │ - Channel    │ │  - RoundRobin (轮流)   │ │
│  │ - 记忆管理    │ │ - 可见性     │ │  - StateMachine (游戏) │ │
│  │ - 工具注册    │ │ - 事件流     │ │  - Hierarchical (委派) │ │
│  └──────────────┘ └──────────────┘ └────────────────────────┘ │
│                                                              │
│  ┌────────────────┐ ┌────────────────┐ ┌────────────────────┐ │
│  │ Memory System  │ │ Observation    │ │ Structured Output  │ │
│  │ - Session 记忆 │ │ - 旁观模式     │ │ - Zod schema 约束  │ │
│  │ - Agent 长记忆 │ │ - 事件重放     │ │ - 投票/决策/审批    │ │
│  │ - 共享产物     │ │ - 历史回溯     │ │ - 防幻觉           │ │
│  └────────────────┘ └────────────────┘ └────────────────────┘ │
├──────────────────────────────────────────────────────────────┤
│                  Layer 1: 基础设施                            │
│  ┌──────────────┐ ┌──────────────┐ ┌────────────────────────┐ │
│  │ LLM Router   │ │ Realtime     │ │ Storage                │ │
│  │ Vercel AI SDK│ │ Socket.io    │ │ Postgres / Redis       │ │
│  │ 多模型统一接口 │ │ / PartyKit   │ │ Agent 持久化           │ │
│  └──────────────┘ └──────────────┘ └────────────────────────┘ │
└──────────────────────────────────────────────────────────────┘
```

4.3 Mode 接口设计（关键抽象）

每个 Mode 是一个插件，实现统一接口：

```
interface Mode {
  id: string                              // "werewolf" | "roundtable" | "trpg" | ...
  name: string                            // 显示名称
  description: string

  // 角色模板 — 定义这个模式下有哪些角色
  roles: RoleTemplate[]                   // e.g. [{id: "wolf", name: "狼人", count: [2,3]}]

  // 流程定义 — 这个模式的状态机/流程
  flowController: FlowController          // FreeForm | RoundRobin | StateMachine

  // 频道配置 — 定义信息隔离规则
  channels: ChannelConfig[]               // e.g. [{id: "wolf-night", roles: ["wolf"], phase: "nigh

  // 结构化输出 — 定义决策点的 schema
  decisionSchemas: Record<string, ZodSchema>  // e.g. {"vote": z.object({target: z.enum(alivePlayer

  // UI 扩展 — 模式特定的 UI 组件
  uiExtensions?: {
    phaseIndicator?: React.FC             // 游戏阶段指示器
    roleCard?: React.FC                   // 角色卡片
    specialActions?: React.FC             // 特殊操作（如女巫的毒药/解药）
  }

  // 生命周期钩子
  onRoomCreated?(room: Room): Promise<void>       // 初始化（分配角色等）
  onPhaseChange?(phase: string): Promise<void>     // 阶段切换
  onMessage?(msg: Message): Promise<void>          // 消息拦截（可用于规则校验）
  onDecision?(decision: Decision): Promise<void>   // 决策回调（投票结果等）
  checkWinCondition?(state: GameState): WinResult | null  // 胜负判定
}
```

4.4 初期目标的难度递增分析

目标	流程控制	信息隔离	角色复杂度	平台核心锻炼点
圆桌辩论	FreeForm + 轮次	无（全公开）	不同立场/模型	Agent 人设、多模型、基础 UI
狼人杀	StateMachine	强（夜/日 + 角色）	固定角色模板	频道系统、状态机、结构化投票
剧本杀	StateMachine + 分支	极强（每人独立线索）	深度人设 + 记忆	文档/线索分发、长记忆、叙事能力
TRPG	半自由（GM 引导）	中等	自由创建 + 成长	GM Agent 推理、骰子系统、叙事生成

圆桌辩论是最佳的 Phase 1 — 它几乎不需要信息隔离和复杂流程，但需要所有平台核心能力：创建房间、配置 Agent 人设、选模型、群聊 UI、旁观。做完这个，平台核心就跑通了。

4.5 Fork vs 自研 — 在通用平台视角下重新评估

通用平台定位让 AgentScope 的吸引力上升（它本身就是通用框架），但核心矛盾依然存在：

维度	Fork AgentScope	自研 (TypeScript)	权重
通用 Agent 编排	8/10 (成熟的 Agent/Msg/Pipeline)	6/10 (需自建核心抽象)	极高

维度	Fork AgentScope	自研 (TypeScript)	权重
通信/信息隔离	9/10 (MsgHub)	7/10 (需自建 Channel 系统)	极高
可插拔模式系统	3/10 (无此概念，需自建)	8/10 (从第一天设计进去)	极高
Web UI 体验	4/10 (需 Python↔JS 桥接)	9/10 (原生 React/Next.js)	极高
多模型支持	9/10 (6+ providers)	9/10 (Vercel AI SDK)	高
记忆系统	8/10 (Working + LongTerm)	5/10 (需自建，但通用场景需求更高)	高
开发速度 (MVP)	5/10 (学框架 + 桥接 + 裁剪)	7/10 (熟悉栈，但核心抽象要设计好)	高
架构演进自由度	4/10 (被框架抽象约束)	9/10 (完全自主)	高
长期维护	3/10 (v2.0 breaking, 2人团队)	8/10 (自己掌控)	高
工具/技能系统	8/10 (Toolkit + MCP)	4/10 (需自建)	中
分布式部署	7/10 (A2A protocol)	3/10 (初期不需要)	低
社区/生态	7/10 (23k stars)	3/10 (从零)	低

加权总分：Fork 5.6 / 自研 7.1

关键差异在于：**"可插拔模式系统"和"UI 体验"的权重极高**，这两项自研远胜。AgentScope 在 Agent 编排和记忆上领先，但这些是"可以追赶的能力"；而模式系统和 UI 体验是"必须从第一天设计对的架构"。

4.6 两条路线详细对比

路线 A: Fork AgentScope (Python backend + TS frontend)

```
你需要做的:
├── Fork agentscope → 裁剪 (去掉 RAG, tuning, audio 等)
├── 新增 Mode 插件系统 (AgentScope 没有此概念)
├── 新增 Room 管理 (AgentScope 没有此概念)
├── 新增 Channel 系统 (MsgHub 可扩展但不等于 Channel)
├── 新建 Web 前端 (React/Next.js)
│   ├── 参考 evotraders 的 RoomView + AgentFeed
│   └── WebSocket 连接 AgentScope 后端
├── WebSocket 桥接层 (Python ↔ JS)
└── 各 Mode 实现 (圆桌/狼人杀/剧本杀/TRPG)

白嫖: Agent 抽象, MsgHub, 记忆系统, 多模型, 工具系统
自建: Mode 系统, Room, Channel, 全部 UI, 桥接层, 所有 Mode
风险: v2.0 breaking change, 双栈摩擦, 框架约束
```

路线 B: 自研 TypeScript 全栈

```
monorepo (Turborepo):
├── apps/web                - Next.js (App Router)
│   ├── 房间创建/加入/配置
│   ├── 群聊 UI (气泡、@mentions、typing indicator)
│   ├── Room View (2D 场景 + agent 头像 + 气泡)
│   ├── 旁观者/上帝视角
│   ├── 角色人设编辑器
│   └── Mode 特定 UI 扩展
│
├── packages/core           - 平台核心 (mode-agnostic)
│   ├── Agent              - 定义、生命周期、人设管理
│   ├── Room               - 房间创建/配置/生命周期
│   ├── Channel            - 频道系统 + 可见性 (借鉴 MsgHub)
│   ├── FlowController     - 可插拔流程 (FreeForm/RoundRobin/StateMachine/Hierarchical)
│   ├── Memory             - Session 记忆 + Agent 长期记忆
│   ├── StructuredOutput   - Zod schema 约束决策
│   └── EventBus           - 平台事件流 (UI 订阅用)
│
├── packages/modes          - Mode 插件
│   ├── roundtable         - 圆桌辩论 (FreeForm + 轮次 + 投票)
│   ├── werewolf           - 狼人杀 (StateMachine + 夜/日/投票)
│   ├── script-kill        - 剧本杀 (StateMachine + 线索 + 分支)
│   ├── trpg               - 跑团 (GM 引导 + 骰子 + 叙事)
│   └── custom             - 用户自定义 (通过配置/prompt 定义规则)
│
├── packages/llm            - Vercel AI SDK wrapper
│   ├── 多模型统一接口 (Claude, GPT, Gemini, Qwen)
│   ├── Streaming
│   └── 结构化输出
│
└── packages/shared         - 类型定义、常量、工具函数

白嫖: Vercel AI SDK (多模型), Socket.io (实时), Next.js (UI)
自建: 平台核心全部, 所有 Mode
优势: 一套栈, 模式系统从第一天设计进去, UI 原生, Vercel 部署
```

5. 多轮反思 — 在通用平台视角下重新验证

第一轮反思：通用平台让 AgentScope 更有吸引力了吗?

质疑： 之前的分析是以"游戏引擎"为前提的。通用平台定位下，AgentScope 作为"通用 agent 框架"的匹配度应该更高了吧?

验证： 确实，AgentScope 的通用能力在平台视角下更有价值： - Agent 抽象（reply/observe 模式）→ 直接可用 - 记忆系统（Working + LongTerm）→ 跨会话人格持久化需要 - 工具系统（Toolkit + MCP）→ 未来通用场景（搜索、写代码等）需要 - A2A 协议 → 分布式部署需要

但关键矛盾没有解决： 1. AgentScope **没有 Mode/插件系统的概念** — 这是通用平台最核心的抽象，需要从头设计 2. AgentScope **没有 Room 的概念** — MsgHub 是代码级的 context manager，不是用户可创建的"房间" 3. **v2.0 重构风险不因定位改变而消失** 4. **Python ↔ JS 桥接的摩擦力不因定位改变而消失**

结论： 通用平台定位让 AgentScope 的 agent 抽象和记忆系统更有吸引力，但最核心的缺失（Mode 系统 + Room + UI）依然存在。净评估从"明显不该 fork"变为"有争议但仍然不建议 fork"。

第二轮反思：自研的记忆系统够用吗？

质疑： 之前说"游戏场景记忆需求轻"，但通用平台需要更强的记忆。Agent 可能需要跨多个房间保持一致人格，甚至"成长"。

验证： 分层分析：

记忆层级	MVP 需求	长期需求	实现复杂度
Session 记忆 (当前房间)	messages[] 数组	同	低
Agent 人设 (持久人格)	system prompt in DB	同 + 自动演化	低→中
Agent 长期记忆 (跨房间)	不需要	关键事件摘要 → DB	中
共享产物 (文档/决策)	不需要	结构化存储	中
语义检索 (recall)	不需要	向量 DB + embedding	高

MVP 阶段只需要前两层（messages[] + system prompt），自建毫无压力。语义检索（AgentScope 的 LongTermMemory）是 Phase 4+ 的事，到时候可以引入 mem0 或者用 Postgres pgvector，不需要现在绑定 AgentScope。

结论： 记忆系统按需演进，不构成现在 fork AgentScope 的理由。

第三轮反思：MsgHub ≠ Channel System，差距有多大？

质疑： 之前说 MsgHub "100-150 行就能实现"，但通用平台的 Channel 系统比 MsgHub 复杂得多。

验证： 确实，通用平台的 Channel 需求远超 MsgHub：

```
MsgHub (AgentScope 现有):
  - 代码级 context manager (开发者写代码控制)
  - 静态订阅 (进入 hub 时确定)
  - 无持久化
  - 无 UI 表示

你需要的 Channel System:
  - 运行时可创建/销毁 (用户或 Mode 动态创建)
  - 动态订阅 (Agent 可以加入/离开频道)
  - 可见性规则 (谁能看到这个频道的存在？)
  - 持久化 (房间状态保存/恢复)
  - UI 表示 (前端需要知道哪些频道存在，当前用户能看到哪些)
  - 嵌套/分组 (频道组 = 一组相关频道)
```

自建 Channel System 的工作量确实不小，大约 300-500 行核心代码 + 类型定义。但这正是平台的核心竞争力 — **这是必须自己理解透彻并完全掌控的模块**，不应该依赖外部框架。

结论： Channel 系统比 MsgHub 复杂，但也比 MsgHub 更重要。越重要的模块越不该外包给第三方。

第四轮反思：Python 真的不行吗？如果游戏只是切入点呢？

质疑： 很多严肃的 AI 项目用 Python。如果未来平台扩展到教育、客服、企业协作等场景，Python 生态的 AI/ML 工具链可能更有价值。

验证： 这是最强的反论。但需要区分两个问题： 1. **LLM 调用** — Vercel AI SDK 在 TypeScript 里已经完全覆盖了 Claude/GPT/Gemini/Qwen，和 Python SDK 能力对等 2. **ML 工具链** (embedding, fine-tuning, 本地模型) — 这些确实 Python 更强。但这些不是 v1 的需求，当真需要时可以用微服务方式引入

更重要的是：**你的产品核心价值是 UI 体验（"建群拉人"的社交隐喻）**。这个无论如何需要出色的前端。TypeScript 全栈在这里的优势是决定性的。

如果硬要用 Python backend，你面对的是： - Python FastAPI/Django + TypeScript Next.js = 两套栈 - 或者用 Python 全栈（Gradio/Streamlit）= UI 体验极差，不可能做出 Accio Work 级别的体验

结论： Python 生态的 AI 工具链优势真实存在，但在 LLM 应用层（你的场景）已被 Vercel AI SDK 抹平。UI 体验的优先级高于 ML 工具链。

第五轮反思：开发顺序对不对？应该从圆桌辩论开始而非狼人杀

质疑： 之前的路线图从狼人杀开始。但通用平台视角下，圆桌辩论更合适作为 Phase 1 —— 它需要平台核心但不需要复杂的游戏机制。

验证： 圆桌辩论 vs 狼人杀作为 Phase 1：

维度	圆桌辩论 Phase 1	狼人杀 Phase 1
需要信息隔离?	不需要	需要（核心）
需要状态机?	简单轮次即可	复杂（夜/日/投票）
需要结构化输出?	可选（投票选最佳观点）	必须（投票选人）
验证平台核心?	Room + Agent + 多模型 + UI	同上 + Channel + StateMachine
用户吸引力?	中（辩论很酷但不够"游戏"）	高（狼人杀人人都懂）
卡兹克已验证?	✅ 圆桌辩论在 Accio Work 上跑通了	❌ 只是概念
开发时间	~1 周	~2-3 周

结论： 圆桌辩论应该是 Phase 1。它在一周内就能出可演示的 demo，验证平台核心（Room、Agent、多模型、UI），而且卡兹克已经证明了这个形态有吸引力。然后 Phase 2 加狼人杀（引入 Channel 和 StateMachine），Phase 3 剧本杀，Phase 4 TRPG。

第六轮反思：有没有可能做一个"两全"方案？

质疑： 核心用 TypeScript，但把 AgentScope 作为一个可选的 "agent runtime backend"，通过 API 调用？不 fork，而是集成。

验证： 这其实是一个有趣的思路： - TypeScript 平台核心 + UI - 当用户需要更强的 Agent 能力（工具、长期记忆、本地模型）时，可以 opt-in 一个 AgentScope backend - 通过 A2A 协议或 REST API 通信

但这带来了新的复杂度： 1. 需要定义平台 ↔ AgentScope 的通信协议 2. 增加部署复杂度 3. 用户体验分裂（"为什么有些功能需要额外安装 Python？"）

结论： 理论上可行但实际增加复杂度，不建议 v1。如果未来真需要 Python ML 能力，以微服务形式引入比现在设计进去更务实。先把 TypeScript 全栈做好。

第七轮反思：拆解"15% 遗憾" — AgentScope 优秀之处的深度分析

记忆系统 — AgentScope 真正优秀的地方

工作记忆压缩（Working Memory Compression）：

AgentScope 的工作记忆会在消息超过阈值时自动调 LLM 做摘要：

```
消息历史 [msg1, msg2, ..., msg50] → 超过阈值
    → LLM 摘要前 40 条 → _compressed_summary = "第一天讨论中，3号被怀疑是狼人..."
    → 后续 prompt = [compressed_summary] + [msg41...msg50]
```

这解决了**长对局 context window 溢出**的真实问题。一局 9 人狼人杀可能 200+ 条消息。没有压缩，要么爆 context window，要么 Agent 丢失早期关键信息。

长期记忆（Long-Term Memory）：

```
Agent 经历关键事件 → record() → 向量化存储 (mem0 / reme)
                                          ↓
后续新会话 → retrieve(query) → cosine similarity 召回相关记忆
                                          ↓
                                  注入 prompt: "你曾经在上一局推理出3号的破绽..."
```

这解决**跨会话人格连续性** — 同一个"侦探"Agent 玩 5 局后应该积累推理经验。

后端可选：InMemory → Redis → SQLAlchemy → Tablestore，从本地到生产全覆盖。

工具系统 — AgentScope 做得不错但可替代

```
# AgentScope 的工具注册
toolkit = Toolkit()
toolkit.add(search_web)        # 自动从类型注解生成 JSON Schema
toolkit.add_mcp("filesystem")  # 一行接入 MCP server
agent = ReActAgent(tools=toolkit)
# → Agent 自动进入 ReAct 循环: Think → Act → Observe → Think...
```

优点：MCP 原生集成、Schema 自动生成、中间件支持、ReAct 循环成熟。

但 TypeScript 已有对等方案：

```
// Vercel AI SDK 的工具注册
const result = await generateText({
  model: anthropic('claude-opus-4-6'),
  tools: {
    searchWeb: tool({
      description: 'Search the web',
      parameters: z.object({ query: z.string() }),  // Zod schema (≈ Pydantic)
      execute: async ({ query }) => await search(query),
    }),
  },
  maxSteps: 10, // ≈ ReAct 循环的 max iterations
})

// MCP 集成
import { Client } from '@modelcontextprotocol/sdk/client'
// → MCP TS 官方 SDK 完全可用
```

按阶段拆解遗憾度

阶段	工作记忆压缩	长期记忆	工具系统	实际遗憾度
Phase 1: 圆桌辩论	不需要（对话短）	不需要	不需要	**0%**
Phase 2: 狼人杀	有用但可绕过（限制轮数）	不需要	不需要	**~3%**
Phase 3: 体验层	同上	不需要	不需要	**~3%**
Phase 4: 剧本杀	**需要**（长对话 + 线索回忆）	有用（跨局侦探直觉）	不需要	**~10%**
Phase 5: TRPG	**需要**	**需要**（角色成长）	有用（GM 查规则书）	**~15%**
Phase 6: 通用平台	需要	需要	**需要**	**~15%**

自建替代方案 — 偷 AgentScope 的设计，用 TypeScript 重写

AgentScope 能力	怎么偷	自建工作量	什么时候做
工作记忆压缩	复刻其逻辑：消息 > 阈值 → 调 LLM 摘要 → 替换旧消息为摘要	~50-80 行 TS	Phase 4 前
长期记忆存储	Postgres + pgvector（Supabase 原生支持）	~100 行 TS + DB schema	Phase 5 前
语义检索召回	pgvector 的 cosine similarity，或 mem0 TS SDK	~80 行 TS	Phase 5 前
工具 Schema 生成	Vercel AI SDK 的 `tool()` + Zod — **已有，零工作量**	0	已有
MCP 集成	`@modelcontextprotocol/sdk` TS 官方 SDK	~30 行 wrapper	Phase 6
ReAct 循环	Vercel AI SDK 的 `maxSteps` — **已有，零工作量**	0	已有

AgentScope 能力	怎么偷	自建工作量	什么时候做
工具中间件	简单 wrapper：`withLogging(tool)`，`withRateLimit(tool)`	~100 行 TS	Phase 6

关键发现：工具系统完全不是遗憾 — Vercel AI SDK + MCP TS SDK 已覆盖。**真正的遗憾集中在记忆系统**，特别是"工作记忆压缩"和"长期语义检索"。但这两个： 1. Phase 1-3 不需要 2. 到 Phase 4-5 时，~200 行代码 + pgvector 可达 AgentScope 80% 效果 3. AgentScope 的实现也不是魔法 — 核心就是"调 LLM 做摘要"+"向量存储 + cosine similarity"

具体"偷"法：记忆压缩的 TypeScript 实现蓝图

从 AgentScope 的 `_working_memory/_base.py` 学到的核心模式：

```
// 借鉴 AgentScope WorkingMemory 的压缩策略
class SessionMemory {
  private messages: Message[] = []
  private compressedSummary: string | null = null
  private readonly maxMessages = 40      // 阈值：超过就压缩
  private readonly keepRecent = 15       // 保留最近 N 条不压缩

  async addMessage(msg: Message) {
    this.messages.push(msg)
    if (this.messages.length > this.maxMessages) {
      await this.compress()
    }
  }

  private async compress() {
    const toCompress = this.messages.slice(0, -this.keepRecent)
    const recent = this.messages.slice(-this.keepRecent)

    // 偷 AgentScope 的核心思路：用 LLM 做摘要
    const summary = await generateText({
      model: anthropic('claude-haiku-4-5-20251001'), // 用便宜模型做摘要
      prompt: `Summarize these conversation messages, preserving all key facts,
               decisions, accusations, and evidence:\n${formatMessages(toCompress)}`,
    })

    this.compressedSummary = this.compressedSummary
      ? `${this.compressedSummary}\n\n${summary.text}`
      : summary.text
    this.messages = recent
  }

  // 给 Agent 的 prompt 用
  getContext(): Message[] {
    const context: Message[] = []
    if (this.compressedSummary) {
      context.push({
        role: 'system',
        content: `[Earlier conversation summary]\n${this.compressedSummary}`
      })
    }
    return [...context, ...this.messages]
  }
}
```

具体"偷"法：长期记忆的 TypeScript 实现蓝图

从 AgentScope 的 `_long_term_memory/` + Stanford Generative Agents 的 reflect 机制：

```
// 借鉴 AgentScope LongTermMemory + Stanford Generative Agents
class AgentLongTermMemory {
  // 记录关键事件（游戏结束后调用）
  async record(agentId: string, event: string, importance: number) {
    const embedding = await embed(event) // OpenAI embedding 或 pgvector 内置
    await db.insert('agent_memories', {
      agent_id: agentId,
      content: event,
      embedding,          // vector(1536)
      importance,         // 0-1, 用 LLM 评估
      created_at: now(),
    })
  }

  // 检索相关记忆（新会话开始时调用）
  async retrieve(agentId: string, context: string, limit = 5): Promise<string[]> {
    const queryEmbedding = await embed(context)
    // pgvector cosine similarity — Supabase 原生支持
    return db.query(`
      SELECT content FROM agent_memories
      WHERE agent_id = $1
      ORDER BY embedding <=> $2  -- cosine distance
      LIMIT $3
    `, [agentId, queryEmbedding, limit])
  }

  // 反思（借鉴 Stanford Generative Agents）
  // 定期让 Agent 回顾近期经历，提炼更高层次的认知
  async reflect(agentId: string) {
    const recentMemories = await this.getRecent(agentId, 20)
    const reflection = await generateText({
      prompt: `Based on these recent experiences, what higher-level insights
               or patterns can you identify about yourself and others?
               ${recentMemories.map(m => m.content).join('\n')}`
    })
    await this.record(agentId, `[Reflection] ${reflection.text}`, 0.9)
  }
}
```

第八轮反思：修正后的最终信心度

决策	信心度	核心依据
不 fork AgentScope	**90%**	v2.0 风险 + 双栈摩擦 + Mode 系统需自建 + 工具系统已有替代
自研 TypeScript 全栈	**90%**	UI 优先 + 单栈效率 + 架构自主 + 记忆系统可追赶
从圆桌辩论开始	**90%**	最快验证核心 + 已被市场验证
Vercel AI SDK 做 LLM 层	**95%**	多模型覆盖 + streaming + structured output + tool use
Phase 4 前自建记忆模块	**95%**	~200 行 TS + pgvector，达到 AgentScope 80% 效果
不用 AgentScope 的遗憾	**10%** → 集中在记忆系统	工具系统已排除，记忆系统可追赶，不是现在 fork 的理由

6. 最终推荐

推荐方案：自研 TypeScript 全栈通用平台，从 AgentScope 偷设计模式

不推荐 fork AgentScope 的核心原因（通用平台视角，按重要性排序）

1. **Mode 插件系统是平台灵魂，AgentScope 没有这个概念** — 需要从头设计，这也意味着你在 AgentScope 上面堆的最核心的代码和它无关
2. **UI 体验是核心竞争力** — Accio Work 的"建群拉人"范式证明了 UI 的价值。Python 后端无法原生支撑出色的前端体验
3. **v2.0 正在大重构** — fork 当前版本是在流沙上建房子
4. **Room 概念需自建** — MsgHub 是代码级 context manager，不是用户可操作的"房间"
5. **双栈摩擦** — 对个人/小团队是每天的消耗
6. **工具系统已有 TypeScript 对等方案** — Vercel AI SDK + MCP TS SDK 完全覆盖

从 AgentScope（及其他项目）偷过来的设计清单

不 fork 不代表不学习。以下是详细的"偷取清单"，按照我们的开发阶段排列：

Phase 1-2 就要偷的（平台核心）

偷自	偷什么	怎么偷	融入位置
AgentScope AgentBase	`reply()` / `observe()` 二元接口	Agent 只需两个方法：生成回复 + 被动接收消息。这个抽象极其干净，直接照搬	`packages/core/agent.ts`
AgentScope MsgHub	嵌套作用域 + auto_broadcast toggle + 动态增删订阅者	Channel 类实现：`new Channel({subscribers, autobroadcast})` 支持嵌套（外层全体 + 内层小组）	`packages/core/channel.ts`
AgentScope Pipeline	Sequential（轮流发言）+ Fanout（并行决策）两个原语	FlowController 的基础操作：`sequential(agents)` 和 `fanout(agents)` 可组合使用	`packages/core/flow.ts`
AgentScope 狼人杀	Pydantic 结构化输出约束投票：`Literal[tuple(alive_players)]` 防幻觉	用 Zod 等价实现：`z.enum(alivePlayers)` 约束投票目标必须是真实存活玩家	`packages/core/structured-output.ts`
AgentScope 狼人杀	Fanout 并行投票 + 关闭 auto_broadcast + 事后批量广播	投票时关闭广播 → 每人独立投 → 收集完毕 → 一次性公布结果	狼人杀 Mode

偷自	偷什么	怎么偷	融入位置
evotraders	WebSocket 推送 + ReadOnlyClient + 心跳重连	前端 WebSocket 客户端：连接、断线重连、心跳、事件分发	`apps/web/lib/realtime.ts`
evotraders	AgentFeed（活动流）+ RoomView（语音气泡 + 头像定位）	前端组件：多 agent 气泡、per-agent 颜色、发言动画	`apps/web/components/`
ChatArena	Arena > Environment > Players 三层抽象	映射为 Platform > Mode > Agent，Mode 定义 Environment 规则	整体架构
Accio Work	"建群拉人"社交隐喻	创建 Agent ≈ 加好友，组团队 ≈ 建群聊，给任务 ≈ 发消息	产品交互设计

Phase 4 前要偷的（记忆系统）

偷自	偷什么	怎么偷	融入位置
AgentScope WorkingMemory	消息超阈值 → LLM 摘要压缩 → 只保留摘要 + 最近消息	上文的 `SessionMemory` 实现，~80 行 TS	`packages/core/memory/session.ts`
AgentScope LongTermMemory	关键事件 → 向量化存储 → 语义检索召回	Postgres pgvector + Supabase，~100 行 TS	`packages/core/memory/long-term.ts`
Stanford Generative Agents	observe → reflect → plan 三步循环	定期让 Agent 回顾经历，提炼高层次认知存入长期记忆	`packages/core/memory/reflect.ts`
ai-murder-mystery	证据系统设计 — 每个 Agent 持有独立线索集	线索作为特殊 Message 类型，绑定到 Agent 的私有 Channel	剧本杀 Mode

Phase 6 可选偷的（通用能力）

偷自	偷什么	怎么偷	融入位置
AgentScope Toolkit	工具中间件（logging, auth, rate limiting）	简单 wrapper 模式：`withLogging(tool)`	`packages/llm/middleware.ts`
AgentScope A2A	跨服务 Agent 通信协议	如果需要分布式部署再考虑	未来
AgentScope 狼人杀 tuner	RL 调优 Agent 游戏策略	有趣但非核心，可作为高级功能	未来

推荐开发路线图

```
Phase 1: 圆桌辩论 — 验证平台核心（~1 周）
  ┌─ 平台核心 ──────────────────────────────────────────┐
  │ Room（创建/加入）+ Agent（人设/模型）+ FreeForm 流程      │
  │ 基础群聊 UI（气泡、@mentions）+ 多模型支持                 │
  └─────────────────────────────────────────────────────┘
  交付：N 个不同模型的 Agent 围坐辩论一个话题
  验证：Room、Agent、LLM 层、基础 UI 全部跑通

Phase 2: 狼人杀 — 引入信息隔离和状态机（~2 周）
  ┌─ 新增 ──────────────────────────────────────────────┐
  │ Channel 系统（频道 + 可见性 mask）                        │
  │ StateMachine FlowController（夜→日→投票→淘汰）            │
  │ StructuredOutput（Zod 约束投票）                          │
  │ UI: 阶段指示器 + 投票面板 + 淘汰动画                       │
  └─────────────────────────────────────────────────────┘
  交付：6-9 个 Agent 完整对局
  验证：信息隔离、状态机、结构化输出

Phase 3: 体验层 — 让人想分享（~1-2 周）
  ┌─ 新增 ──────────────────────────────────────────────┐
  │ Room View 可视化（参考 evotraders）                       │
  │ 角色人设编辑器 + AI 自动丰富                               │
  │ 人类玩家加入（Human Agent）                                │
  │ 旁观者/上帝视角模式                                        │
  └─────────────────────────────────────────────────────┘
  验证：体验足够好，用户愿意截图分享

Phase 4: 剧本杀 — 深度叙事（~2-3 周）
  ┌─ 新增 ──────────────────────────────────────────────┐
  │ 线索/证据分发系统                                          │
  │ 分支叙事引擎（剧本杀的核心）                               │
  │ Agent 长期记忆（跨场景的角色一致性）                       │
  │ 剧本模板系统（用户可上传/分享剧本）                        │
  └─────────────────────────────────────────────────────┘

Phase 5: TRPG — 开放世界叙事（~3 周）
  ┌─ 新增 ──────────────────────────────────────────────┐
  │ GM Agent（有 LLM 推理能力的游戏主持人）                    │
  │ 骰子系统 + 技能检定                                        │
  │ 叙事生成（observe → reflect → plan，借鉴 Stanford）       │
  │ 角色成长系统（角色卡 + 属性 + 装备）                       │
  └─────────────────────────────────────────────────────┘

Phase 6: 平台化 — 通用场景（~2 周）
  ┌─ 新增 ──────────────────────────────────────────────┐
  │ 自定义 Mode（用户通过配置/prompt 定义规则）                │
  │ Agent 人设市场（社区分享角色）                             │
  │ 房间模板市场（社区分享场景）                               │
  │ 游戏回放系统                                               │
  │ Hierarchical FlowController（三省六部式委派）              │
  └─────────────────────────────────────────────────────┘
```

推荐技术栈总结

```
monorepo (Turborepo)
├── apps/web              — Next.js 15 (App Router), Tailwind, shadcn/ui
├── packages/core         — 平台核心: Agent, Room, Channel, FlowController, Memory, EventBus
├── packages/modes        — Mode 插件: roundtable, werewolf, script-kill, trpg, custom
├── packages/llm          — Vercel AI SDK (Claude, GPT, Gemini, Qwen) + structured output
├── packages/shared       — 类型定义, 常量
└── 实时通信              — Socket.io (初期) / PartyKit (如果需要 edge)
    存储                  — Postgres (Supabase) + Redis (可选)
    部署                  — Vercel
```

附录 A：所有调研项目完整列表

通用多 Agent 框架

- AgentScope: https://github.com/agentscope-ai/agentscope (23.5k ⭐)
- AgentScope Samples: https://github.com/agentscope-ai/agentscope-samples (274 ⭐)
- HiClaw: https://github.com/agentscope-ai/HiClaw (4k ⭐)
- ClawTeam: https://github.com/HKUDS/ClawTeam (4.7k ⭐)
- MetaGPT: https://github.com/FoundationAgents/MetaGPT (67k ⭐)
- CAMEL: https://github.com/camel-ai/camel (16.7k ⭐)
- AgentVerse: https://github.com/OpenBMB/AgentVerse (5k ⭐)

游戏/模拟专用

- ChatArena (deprecated): https://github.com/Farama-Foundation/chatarena (1.5k ⭐)
- Google Werewolf Arena: https://github.com/google/werewolf_arena (46 ⭐)
- Stanford Generative Agents: https://github.com/joonspk-research/generative_agents (21k ⭐)
- Danghuangshang: https://github.com/wanikua/danghuangshang (2.6k ⭐)

剧本杀 / 跑团

- ai-murder-mystery: https://github.com/ScottishFold007/ai-murder-mystery (16 ⭐)
- jubensha-ai: https://github.com/JianWang97/jubensha-ai (89 ⭐)
- dnd-llm-game: https://github.com/tegridydev/dnd-llm-game (101 ⭐)
- AI-DungeonMaster: https://github.com/nickwalton/AI-DungeonMaster (77 ⭐)
- GameMasterAI: https://github.com/deckofdmthings/GameMasterAI (29 ⭐)
- ChatRPG: https://github.com/KarmaKamikaze/ChatRPG (29 ⭐)
- XTalk (RPGGO): https://github.com/RPGGO-AI/XTalk (15 ⭐)
- ZRIC-AI-TRPG-Engine: https://github.com/zRICGao/ZRIC-AI-TRPG-Engine (3 ⭐)
- TRPG-Agent: https://github.com/wenliang8102/TRPG-Agent (2 ⭐)

闭源参考

- Accio Work: https://www.accio.com/work (阿里国际站)